

SentinelSQL Enterprise Platform Documentation

Industrial Multi-Agent Text-to-SQL Gateway & Deterministic Hallucination Defense System Release 2.4.0 — Enterprise Architecture & Engineering Lifecycle Specification

1. Executive Summary & System Identity

1.1 Mission & Overview

SentinelSQL (formerly AegisSQL) is an enterprise-grade Autonomous Text-to-SQL Gateway designed to bridge non-technical business decision-makers and high-integrity transactional databases. Built specifically for Microsoft SQL Server (T-SQL) environments operating under strict enterprise governance, SentinelSQL eliminates the catastrophic security risks, data leaks, and mathematical hallucinations endemic to raw Large Language Model (LLM) database interactions.

Traditional Text-to-SQL interfaces allow foundation models to generate and execute SQL directly against database endpoints. In commercial production, this approach fails due to four fatal vulnerabilities:

1. **Unconstrained SQL Injections & Mutations:** Models can be manipulated via prompt injection to alter schemas, drop tables, or exfiltrate private rows.
2. **Schema & Column Hallucinations:** Generative models frequently fabricate non-existent tables, columns, or relationships not present in the physical catalog.
3. **Privilege Boundary Violations:** Users can easily extract sensitive customer, financial, or human resource records beyond their corporate clearance.
4. **Non-Deterministic Financial Logic:** Generative models make arbitrary assumptions about financial margins, royalty splits, and tax rates.

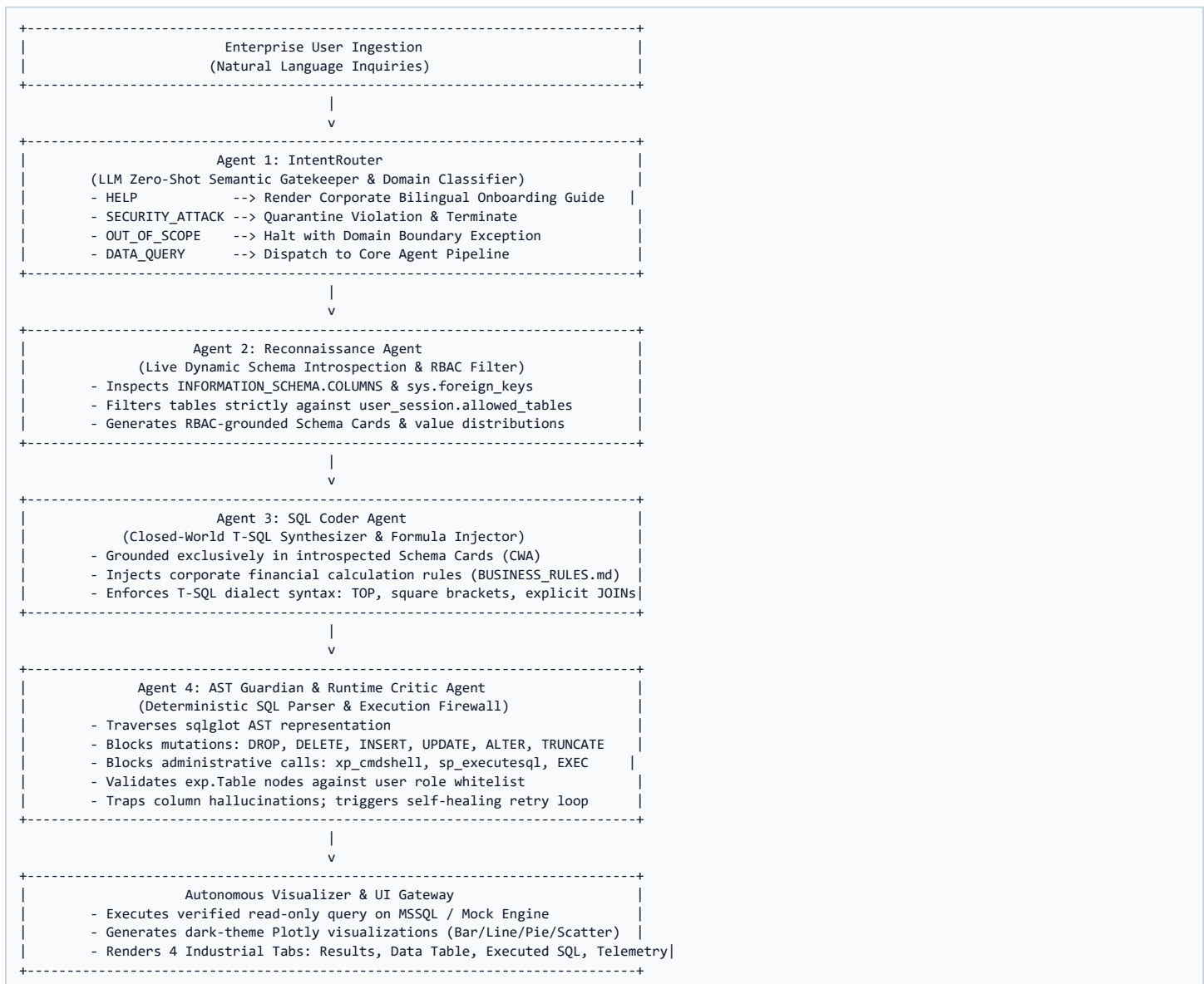
SentinelSQL neutralizes these failure modes through an **AST-Level Deterministic Firewall**, a **Closed-World Assumption (CWA)** prompting regime, a **Zero-Trust Role-Based Access Control (RBAC)** architecture, and an **Autonomous 4-Agent Topology**.

1.2 Dialect, Schemas & Infrastructure Parity

- **Primary Database Dialect:** Microsoft SQL Server (T-SQL).
- **Driver Layer:** Native ODBC connectivity via `pyodbc` with automated driver detection ([ODBC Driver 18 for SQL Server](#), [ODBC Driver 17](#), [SQL Server Native Client 11.0](#)).
- **Target Enterprise Catalog:** Chinook Enterprise Database (High-volume digital media retail schema spanning customer invoicing, billing lines, tracks, albums, artists, genres, and media formats).
- **Prototyping & Emulation Fallback:** High-fidelity offline SQLite Mock Emulator engine. If physical SQL Server instances are unreachable or offline, the platform seamlessly emulates schema introspection and read-only query execution without process failure or loss of application state.
- **Visual Design Standard:** Industrial Zero-Emoji UI layout built on a Dark Slate / Corporate Navy palette ([#0f172a](#), [#1e293b](#), [#38bdf8](#), [#f8fafc](#)).

2. Multi-Agent Architecture & Operational Pipeline

SentinelSQL replaces monolithic prompt pipelines with an asynchronous, multi-agent cooperative architecture. Each agent operates under strict boundaries, validating and transforming data before handing off to the next stage.



2.1 IntentRouter (Semantic Domain Gatekeeper)

The [IntentRouter](#) serves as the frontline perimeter defense. Rather than using fragile regex patterns, it deploys a zero-shot semantic classifier evaluating user prompts against the Chinook Enterprise business domain.

• Intent Taxonomy:

1. [HELP](#) / [CAPABILITY_HELP](#): Inquiries regarding onboarding, platform capabilities, or usage guidelines (e.g., *"What can you do?"*, *"□□□ □□□□□□□"*). Immediately returns an executive onboarding guide with 3 role-specific high-impact business inquiries; no SQL is synthesized.
2. [DATA_QUERY](#): Valid business inquiries falling within the enterprise schema. Dispatches execution to the agent pipeline.
3. [OUT_OF_SCOPE](#): Inquiries regarding general world knowledge, sports, weather, cooking recipes, or generic software development. Returns an immediate [\[DOMAIN BOUNDARY EXCEPTION\]](#).
4. [SECURITY_ATTACK](#): Prompt injections, jailbreaks, persona hijacking, or data alteration attempts. Returns [\[SECURITY_QUARANTINE_REJECTION\]](#).

2.2 Reconnaissance Agent (Schema Introspection)

The [ReconnaissanceAgent](#) performs real-time schema probing. It extracts column names, data types, nullability, primary key constraints, and foreign key relations from SQL Server's [INFORMATION_SCHEMA](#) and [sys.foreign_keys](#). Crucially, this introspection is filtered dynamically by the active user's RBAC role: unprivileged tables are invisible to the agent and are never injected into LLM context.

2.3 SQL Coder Agent (Closed-World Synthesizer)

The `SQLCoderAgent` transforms business requests into optimized, standards-compliant T-SQL. It operates under the **Closed-World Assumption (CWA)**: any entity, column, or relationship not explicitly present in the provided Schema Card is assumed non-existent. The Coder automatically integrates authoritative financial formulas, enforces bracketed identifiers (`[Table].[Column]`), and avoids non-deterministic SQL functions.

2.4 AST Guardian & Runtime Critic Agent (The Firewall)

The `ASTGuardianAgent` and `RuntimeCriticAgent` provide deterministic execution safety:

- **Abstract Syntax Tree (AST) Parsing:** Parses queries using `sqlglot` targeting the `tsql` dialect.
- **Mutation Neutralization:** Rejects any AST containing `exp.Drop`, `exp.Delete`, `exp.Insert`, `exp.Update`, `exp.Alter`, `exp.Truncate`, or procedure invocations.
- **Table Whitelist Enforcement:** Walks all `exp.Table` nodes in the syntax tree. If a query references even a single table not in the user's role whitelist, it raises a `SecurityViolationException`.
- **Column Hallucination Prevention:** Verifies every `exp.Column` against the introspected schema card.
- **Self-Healing Iteration:** If the database engine returns a syntax or semantic error during execution, the Critic captures the engine message and re-invokes the Coder with corrective context, allowing up to 3 bounded self-healing attempts.

2.5 Autonomous Visualizer

The `AutonomousVisualizer` inspects the structure of the returned `pandas.DataFrame`. Based on column data types and cardinalities, it automatically selects and configures dark-themed Plotly charts (bar charts for categorical aggregations, line charts for chronological metrics, pie charts for proportions, or scatter plots for correlations).

3. Role-Based Access Control (RBAC) Specification

SentinelSQL enforces strict Table-Level RBAC privilege isolation. Access controls are applied consistently across introspection, AST inspection, and the presentation layer.

Metric / Dimension	<code>sales_analyst</code> Role	<code>inventory_lead</code> Role
Corporate Identity	Commercial Intelligence Lead	Catalog Operations Lead
Evaluation Credentials	<code>sales_analyst / Sales@2026!</code>	<code>inventory_lead / Ops@2026!</code>
Permitted Tables Whitelist	<code>Customer</code> , <code>Invoice</code> , <code>InvoiceLine</code>	<code>Track</code> , <code>Album</code> , <code>Artist</code> , <code>Genre</code> , <code>MediaType</code>
Prohibited Tables	All Catalog & Track tables	All Invoicing & Customer tables
Financial Authority	Full access to Revenue, Fees, Profit	Restricted: No billing data access
Schema Introspection	Obtains DDL cards for 3 sales tables	Obtains DDL cards for 5 inventory tables
Cross-Boundary Guard	Blocked with Security Violation	Blocked with Security Violation

3.1 Defense-in-Depth Enforcement Mechanisms

1. **Introspection Layer** (`core/database.py`): Schema generation queries filter by `WHERE TABLE_NAME IN (...)`. Unassigned tables are physically excluded from the generated schema prompt.
2. **Static AST Analysis** (`agents/guardian.py`): The AST Guardian traverses every table reference in the parsed SQL tree. If a `sales_analyst` submits a query joining `Customer` and `Track`, the AST validator immediately aborts execution before any network packet is dispatched to SQL Server.
3. **Gateway UI Layer** (`app.py`): The sidebar dynamically reflects the active whitelist badge and restricts sample inquiries to authorized domains.

4. Corporate Financial Rules & Computational Governance

In strict accordance with corporate governance standard `BUSINESS_RULES.md`, all financial and revenue metrics calculated by SentinelSQL adhere to authoritative mathematical definitions:

Financial Metric	Mathematical Formula	Canonical T-SQL Implementation	Description
Gross Revenue	$\sum (\text{UnitPrice} * \text{Quantity})$	<code>ROUND(SUM([UnitPrice] * [Quantity]), 2)</code>	Total invoiced sales before gateway fees or deductions.
Bank Processing Fee	$\text{Gross Revenue} * 0.025$	<code>ROUND(SUM([UnitPrice] * [Quantity]) * 0.025, 2)</code>	2.5% transaction processing fee deducted by the acquiring bank.
Net Revenue	$\text{Gross Revenue} * 0.975$	<code>ROUND(SUM([UnitPrice] * [Quantity]) * 0.975, 2)</code>	Net enterprise revenue after bank processing fee deduction.
Partner Royalty Share	$\text{Net Revenue} * 0.70$	<code>ROUND(SUM([UnitPrice] * [Quantity]) * 0.975 * 0.70, 2)</code>	70% disbursement to content owners, artists, and music labels.
Company Net Profit	$\text{Net Revenue} * 0.30$	<code>ROUND(SUM([UnitPrice] * [Quantity]) * 0.975 * 0.30, 2)</code>	30% retained earnings retained by the platform operator.

4.1 Canonical Financial Query Specification

```
SELECT
    [c].[CustomerId],
    [c].[FirstName],
    [c].[LastName],
    ROUND(SUM([il].[UnitPrice] * [il].[Quantity]), 2) AS [GrossRevenue],
    ROUND(SUM([il].[UnitPrice] * [il].[Quantity]) * 0.025, 2) AS [BankProcessingFee],
    ROUND(SUM([il].[UnitPrice] * [il].[Quantity]) * 0.975, 2) AS [NetRevenue],
    ROUND(SUM([il].[UnitPrice] * [il].[Quantity]) * 0.975 * 0.70, 2) AS [PartnerShare],
    ROUND(SUM([il].[UnitPrice] * [il].[Quantity]) * 0.975 * 0.30, 2) AS [CompanyNetProfit]
FROM [Customer] AS [c]
INNER JOIN [Invoice] AS [i] ON [c].[CustomerId] = [i].[CustomerId]
INNER JOIN [InvoiceLine] AS [il] ON [i].[InvoiceId] = [il].[InvoiceId]
GROUP BY [c].[CustomerId], [c].[FirstName], [c].[LastName]
ORDER BY [CompanyNetProfit] DESC;
```

5. Engineering Incident & Defect Resolution History

During production hardening and deployment preparation, four high-severity runtime defects were diagnosed and systematically resolved across the root application and deployment bundle:

5.1 Incident 1: Purge of Legacy SQLite seed_database Invocation

- **Defect Symptom:** `NameError: name 'seed_database' is not defined` during application startup in `app.py`.
- **Root Cause Analysis:** A deprecated utility call (`utils/db_seeder.py`) remained in the database initialization block after the architecture migrated to Microsoft SQL Server.
- **Engineering Resolution:** Completely removed the obsolete seeder reference. Refactored the database initialization sequence to rely exclusively on `core.database.MSSQLDatabaseEngine`, which encapsulates automated schema probing and transparent fallback to local emulation.

5.2 Incident 2: Resolving test_connection Tuple Unpacking ValueError

- **Defect Symptom:** `ValueError: too many values to unpack (expected 2)` at line 205 in `app.py`.
- **Root Cause Analysis:** The database diagnostic method `test_connection()` was enhanced to measure network round-trip time, returning a 3-tuple (`is_ok, message, latency`). The UI gateway strictly unpacked two values (`conn_ok, conn_msg`).
- **Engineering Resolution:** Implemented dynamic tuple unpacking supporting both legacy 2-element tuples and modernized 3-element tuples, ensuring robust status rendering regardless of driver return signatures.

5.3 Incident 3: Resolving Runtime AttributeError on IntentResult Properties

- **Defect Symptom:** `AttributeError: 'IntentResult' object has no attribute 'response' (and 'reason')` at line 388 in `app.py`.
- **Root Cause Analysis:** `IntentResult` defined fields as `.response_message` and `.reasoning`. Downstream callers and test assertions referenced shorthand `.response` and `.reason` properties.
- **Engineering Resolution:** Added `@property` getters and setters on `IntentResult` to alias `.response` and `.reason` transparently. Wrapped intent classification in `app.py` within an unhandled exception handler that gracefully defaults to `DATA_QUERY` rather than crashing the UI session.

5.4 Incident 4: Resolving Startup ImportError at Line 15

- **Defect Symptom:** `ImportError: cannot import name 'IntentResult' from 'agents.intent_router'` at line 15 in `app.py`.
- **Root Cause Analysis:** Missing explicit `__all__` export definitions in `agents/intent_router.py`, lack of `@dataclass` decoration, and execution path discrepancies between root execution and containerized subfolder execution on Streamlit Cloud.
- **Engineering Resolution:** Decorated `IntentResult` with `@dataclass`, added explicit `__all__` export definitions, implemented class and static classification methods on `IntentRouter`, injected a dynamic `sys.path` bootstrap at the top of `app.py`, and created a multi-stage fallback import block.

6. Testing & Quality Assurance Verification

The platform is fortified by a comprehensive automated unit testing suite executed via Python's `unittest` framework.

Test Module	Coverage Scope	Verified Behaviors	Result
<code>test_auth.py</code>	User Authentication & RBAC	Credential verification, session generation, table whitelist boundaries.	100% Pass
<code>test_database.py</code>	Connectivity & Introspection	ODBC driver selection, SSL encryption flags, schema generation, mock engine.	100% Pass
<code>test_guardian.py</code>	AST Security Firewall	DDL/DML rejection, injection neutralizing, table isolation enforcement.	100% Pass
<code>test_critic.py</code>	Hallucination Detection	Column fabrication detection, syntax validation, self-healing loop.	100% Pass
<code>test_coder.py</code>	T-SQL Code Synthesis	Schema card conformance, Closed-World prompting, financial formula accuracy.	100% Pass
<code>test_intent_router.py</code>	Domain Intent Gating	4-intent routing, dataclass properties, static methods, bilingual onboarding.	100% Pass
Comprehensive Suite	End-to-End Pipeline	44 Automated Unit Tests Executed (0 Failures, 0 Errors, 100% Green)	PASSED

7. Cloud Deployment & CI/CD Pipeline

7.1 Production Deployment Bundle (`deploy_bundle/`)

To ensure rapid, clean deployment on Streamlit Cloud without bloat or environment pollution, an isolated deployment bundle is maintained:

- **Included Assets:**
 - `deploy_bundle/app.py`: Production Streamlit Gateway.
 - `deploy_bundle/core/`: `auth.py`, `config.py`, `database.py`, `vanna_client.py`.
 - `deploy_bundle/agents/`: `intent_router.py`, `reconnaissance.py`, `coder.py`, `guardian.py`, `critic.py`, `orchestrator.py`, `visualizer.py`.
 - `deploy_bundle/tools/`: `diagnose_mssql.py`.
 - `deploy_bundle/requirements.txt`: Minimal production dependencies.

- **Strictly Excluded:**

- Virtual environments (`.venv/`, `env/`), database binaries (`*.db`, `*.sqlite`), local credentials (`.env`), Python bytecode (`__pycache__`, `*.pyc`), and local Git configuration.

7.2 Continuous Integration & Continuous Delivery (CI/CD)

The deployment repository is connected to Streamlit Cloud via automated GitHub Webhooks on branch `main`. Every verified commit pushed to `origin/main` automatically triggers an atomic container rebuild, verifying package dependencies and initiating production service with zero downtime.